

Relatório - MC714 Trabalho 2

Nome: Caio Zavarezzi

RA: 195260

Descrição geral

Este trabalho foi desenvolvido para a disciplina MC714 - Sistemas Distribuídos. A proposta era implementar algoritmos distribuídos usando alguma forma real de comunicação entre processos.

Neste projeto, eu implementei um sistema simples com vários nós rodando localmente. Cada nó é executado como um processo Python separado e se comunica com os outros nós usando sockets TCP. As mensagens trocadas entre os processos são enviadas em formato JSON.

Foram implementados três algoritmos principais:

1. Relógio lógico de Lamport
2. Exclusão mútua distribuída com Ricart-Agrawala
3. Eleição de líder com algoritmo Bully

A ideia foi montar uma implementação simples, mas que desse para observar claramente a troca de mensagens entre os nós e o funcionamento dos algoritmos no terminal.

Problema abordado

Em sistemas distribuídos, os processos não compartilham memória e geralmente executam de forma independente. Por isso, alguns problemas aparecem naturalmente.

Um desses problemas é a ordenação de eventos. Como cada processo tem sua própria execução, não dá para depender apenas de um relógio físico global para saber a ordem dos acontecimentos. Para isso, foi usado o relógio lógico de Lamport.

Outro problema é o acesso a uma região crítica. Em um sistema distribuído, pode existir um recurso que não deve ser acessado por mais de um processo ao mesmo tempo. Para tratar isso, foi implementado o algoritmo de exclusão mútua de Ricart-Agrawala.

Também existe o problema de escolher um coordenador ou líder entre os nós ativos. Para isso, foi implementado o algoritmo Bully, em que o nó ativo com maior ID assume como líder.

Ambiente de implementação

A implementação foi feita em Python 3.

Foram usadas apenas bibliotecas padrão da linguagem:

- socket: usada para comunicação TCP entre os processos;
- threading: usada para permitir que o nó receba mensagens enquanto também aceita comandos digitados no terminal;
- json: usada para estruturar as mensagens enviadas entre os nós;
- sys: usada para ler o ID do nó pela linha de comando.

A execução foi feita localmente, com cada nó rodando em um terminal diferente. Mesmo estando na mesma máquina, cada nó possui sua própria porta TCP e se comunica com os demais por troca de mensagens.

Estrutura do projeto

O projeto foi organizado com os seguintes arquivos principais:

- node.py: arquivo principal da implementação. Ele contém a lógica de comunicação, o relógio de Lamport, a exclusão mútua e a eleição de líder.
- config.py: arquivo com a configuração dos nós e das portas usadas por cada processo.
- README.md: arquivo com instruções para executar o projeto e testar os algoritmos.
- relatorio.md: relatório explicando a implementação, os testes e as fontes utilizadas.

No arquivo config.py, os nós são configurados da seguinte forma:

```
NODES = {  
1: ("localhost", 5001),  
2: ("localhost", 5002),  
3: ("localhost", 5003),  
}
```

Com essa configuração, o Nó 1 escuta na porta 5001, o Nó 2 escuta na porta 5002 e o Nó 3 escuta na porta 5003.

Para executar os nós, são usados comandos em terminais separados:

```
python node.py 1
```

```
python node.py 2
```

```
python node.py 3
```

Cada terminal representa um processo independente do sistema distribuído.

Comunicação entre processos

Cada nó, ao ser iniciado, abre um socket TCP e fica escutando conexões na porta definida no arquivo config.py.

Além disso, foi usada uma thread para que o nó consiga receber mensagens pela rede ao mesmo tempo em que aceita comandos digitados pelo usuário no terminal. Isso foi necessário porque, sem thread, o programa ficaria preso apenas esperando conexões e não seria possível digitar comandos no terminal.

As mensagens são enviadas em formato JSON. Um exemplo de mensagem comum é:

```
{
  "type": "MESSAGE",
  "sender": 1,
  "clock": 1,
  "content": "primeira mensagem"
}
```

Os campos principais das mensagens são:

- type: indica o tipo da mensagem;
- sender: indica qual nó enviou a mensagem;
- clock: carrega o valor do relógio lógico de Lamport;
- content: guarda o conteúdo textual da mensagem.

Durante a implementação, foram usados vários tipos de mensagem:

- MESSAGE: mensagem comum entre nós;
- REQUEST: pedido para entrar na região crítica;
- REPLY: resposta dando permissão para entrada na região crítica;
- ELECTION: mensagem usada para iniciar uma eleição de líder;
- OK: resposta de um nó ativo durante a eleição;
- COORDINATOR: mensagem informando quem é o novo líder.

Essa estrutura com JSON facilitou bastante a implementação, porque cada nó consegue identificar o tipo da mensagem recebida e agir de acordo com esse tipo.

Relógio lógico de Lamport

Cada nó possui uma variável local chamada lamport_clock.

Esse relógio começa em zero e é atualizado de acordo com as regras do relógio lógico de Lamport.

Quando um nó envia uma mensagem, ele incrementa seu relógio antes do envio.

Quando um nó recebe uma mensagem, ele atualiza seu relógio usando a regra:

$$\text{clock_local} = \max(\text{clock_local}, \text{clock_recebido}) + 1$$

Por exemplo, se o Nó 1 envia uma mensagem com clock 1 para o Nó 2, e o Nó 2 estava com clock 0, então o Nó 2 atualiza seu relógio para:

$$\max(0, 1) + 1 = 2$$

Isso permite criar uma ordem lógica entre eventos distribuídos, mesmo sem usar relógios físicos sincronizados.

Na implementação, o valor do relógio é mostrado no terminal tanto no envio quanto no recebimento das mensagens. Assim, durante os testes, é possível observar o clock avançando conforme os nós se comunicam.

Exclusão mútua distribuída

Para a parte de exclusão mútua, foi implementado o algoritmo de Ricart-Agrawala.

A região crítica representa uma parte do sistema que só pode ser acessada por um nó de cada vez. No projeto, ela foi representada por mensagens no terminal indicando quando um nó entrou ou saiu da região crítica.

Quando um nó deseja entrar na região crítica, ele executa o comando:

`request_cs`

Ao fazer isso, o nó:

1. incrementa seu relógio lógico;
2. guarda o timestamp do pedido;
3. envia mensagens REQUEST para os outros nós;
4. aguarda receber REPLY de todos os demais;
5. entra na região crítica somente depois de receber todas as permissões.

A prioridade entre pedidos é definida pelo par:

`(timestamp, node_id)`

O menor timestamp tem prioridade. Se houver empate no timestamp, o menor ID vence.

Quando um nó recebe um REQUEST, ele verifica sua própria situação. Se ele já está dentro da região crítica, ele adia a resposta. Se ele também está tentando entrar na região crítica e tem prioridade maior, ele também adia a resposta. Caso contrário, ele responde imediatamente com REPLY.

Quando um nó sai da região crítica usando o comando:

`release_cs`

ele libera as respostas que tinham ficado pendentes.

Essa lógica garante que dois nós não entrem na região crítica ao mesmo tempo.

Eleição de líder com Bully

Para a eleição de líder, foi implementado o algoritmo Bully.

A ideia usada foi que o nó ativo com maior ID deve ser o líder.

Com três nós configurados, o líder inicial é o Nó 3, porque ele possui o maior ID.

O comando:

`leader`

mostra qual líder o nó conhece no momento.

Para iniciar uma eleição, foi criado o comando:

`election`

Quando um nó inicia uma eleição, ele tenta enviar mensagens ELECTION para todos os nós com ID maior que o seu.

Se algum nó maior estiver ativo, ele pode responder com OK. Se nenhum nó maior responder, o nó que iniciou a eleição assume como novo líder e envia uma mensagem COORDINATOR para os outros nós.

A falha do líder foi simulada encerrando o processo do Nó 3 com CTRL + C. Depois disso, o Nó 2 iniciou a eleição. Como ele não conseguiu contato com o Nó 3, ele assumiu como novo líder e notificou o Nó 1.

Essa implementação é uma versão simples do Bully, feita para demonstrar a ideia principal do algoritmo.

Comandos implementados

Durante a execução de cada nó, os principais comandos disponíveis são:

`send <id_do_destino>`

Esse comando envia uma mensagem comum para outro nó.

`request_cs`

Esse comando solicita entrada na região crítica.

`release_cs`

Esse comando libera a região crítica.

`leader`

Esse comando mostra qual líder o nó conhece atualmente.

`election`

Esse comando inicia uma eleição de líder.

`exit`

Esse comando encerra o nó.

Esses comandos foram usados nos testes e também na demonstração em vídeo.

Testes realizados

Teste de comunicação e Lamport

Primeiro, foi testada a comunicação simples entre dois nós.

Com o Nó 1 e o Nó 2 em execução, foi executado no Nó 1:

`send 2 primeira mensagem`

O Nó 1 incrementou seu relógio lógico antes de enviar a mensagem.

A saída esperada no Nó 1 foi semelhante a:

Nó 1 enviou mensagem para Nó 2 | `clock=1`

No Nó 2, a mensagem foi recebida e o relógio lógico foi atualizado:

Nó 2 recebeu MESSAGE de Nó 1: primeira mensagem | `clock recebido=1` | `clock atual=2`

Esse teste mostrou que a comunicação por socket estava funcionando e que o relógio de Lamport estava sendo atualizado corretamente no recebimento.

Também foi testado o envio de uma resposta do Nó 2 para o Nó 1, para verificar se os clocks continuavam avançando corretamente.

Teste de exclusão mútua

Depois, foi testada a entrada na região crítica.

Com três nós rodando, foi executado no Nó 1:

`request_cs`

O Nó 1 enviou mensagens REQUEST para os nós 2 e 3.

Os nós 2 e 3 responderam com REPLY.

Depois de receber todas as respostas, o Nó 1 entrou na região crítica.

A saída observada foi semelhante a:

*** Nó 1 ENTROU na região crítica ***

Depois, foi executado:

release_cs

E o Nó 1 saiu da região crítica:

*** Nó 1 SAIU da região crítica ***

Esse teste mostrou que um nó só entra na região crítica depois de receber permissão dos outros nós.

Teste de disputa pela região crítica

Também foi testado um caso em que dois nós tentam entrar na região crítica.

Nesse teste, foram executados comandos request_cs em dois nós diferentes.

O algoritmo usou o timestamp de Lamport e o ID do nó para decidir a prioridade.

O nó com maior prioridade entrou primeiro na região crítica. O outro ficou esperando. Depois que o primeiro nó executou release_cs, o segundo pôde entrar.

Esse teste foi importante porque mostrou que a exclusão mútua estava sendo preservada. Ou seja, dois nós não entraram na região crítica ao mesmo tempo.

Teste de eleição de líder

Por fim, foi testado o algoritmo Bully.

Com os três nós rodando, foi executado o comando:

leader

O resultado indicou que o líder inicial era o Nó 3.

Depois, o processo do Nó 3 foi encerrado com CTRL + C, simulando uma falha do líder.

No Nó 2, foi executado:

election

O Nó 2 tentou se comunicar com o Nó 3, mas não conseguiu. Como não havia nenhum nó maior ativo, o Nó 2 se declarou novo líder.

A saída observada no Nó 2 foi semelhante a:

Nó 2 iniciou eleição.
Nó 2 não conseguiu contato com Nó 3
*** Nó 2 virou o novo LÍDER ***
Nó 2 enviou COORDINATOR para Nó 1

No Nó 1, apareceu a mensagem indicando que ele reconheceu o Nó 2 como novo líder.

Esse teste demonstrou o funcionamento básico do algoritmo Bully após a falha simulada do líder.

Resultados obtidos

Os testes realizados mostraram que a implementação conseguiu atender aos principais objetivos do trabalho.

Foi possível observar:

- os nós rodando como processos independentes;
- a comunicação por sockets TCP;
- o envio de mensagens em JSON;
- a atualização do relógio lógico de Lamport;
- o controle de entrada e saída da região crítica;
- a disputa entre nós pela região crítica;
- a eleição de um novo líder após a falha simulada do líder anterior.

A execução local em múltiplos terminais ajudou a visualizar a troca de mensagens entre os processos e facilitou a demonstração dos algoritmos.

Limitações da implementação

A implementação foi feita com objetivo didático e para demonstração dos algoritmos.

Algumas limitações são:

- os nós foram executados localmente, embora usando comunicação real via sockets TCP;
- a falha do líder foi simulada manualmente com CTRL + C;
- a eleição é iniciada manualmente pelo comando election;
- não foi implementado heartbeat periódico para detectar falhas automaticamente;
- não há persistência de estado caso os processos sejam encerrados;
- a região crítica foi demonstrada por mensagens no terminal, e não por um recurso externo complexo.

Mesmo com essas limitações, a implementação demonstra os conceitos principais pedidos no trabalho.

Fontes, apoio utilizado e alterações feitas

Durante o desenvolvimento do trabalho, eu não utilizei um projeto pronto retirado da Internet como base direta da implementação. O código foi sendo construído aos poucos, a partir dos requisitos do enunciado e dos testes feitos localmente com os nós rodando em terminais separados.

As principais referências usadas foram conceituais, para entender melhor os algoritmos implementados:

- relógios lógicos de Lamport;
- exclusão mútua distribuída;
- algoritmo de Ricart-Agrawala;
- eleição de líder;
- algoritmo Bully;
- comunicação por sockets TCP;
- mensagens em formato JSON.

Também utilizei o ChatGPT como ferramenta de apoio durante o desenvolvimento. O uso foi principalmente para tirar dúvidas sobre os algoritmos, organizar uma ordem de implementação, entender erros que apareceram nos testes e revisar partes do código. Durante o desenvolvimento, algumas decisões e alterações importantes foram feitas:

1. Primeiro, foi criada uma comunicação simples entre processos usando sockets TCP. A ideia era fazer cada nó conseguir escutar em uma porta e enviar mensagens para outro nó.
2. Depois, as mensagens deixaram de ser texto puro e passaram a ser enviadas em formato JSON. Isso facilitou a identificação do tipo da mensagem, do nó remetente, do relógio lógico e do conteúdo.
3. Em seguida, foi implementado o relógio lógico de Lamport. Para isso, cada nó passou a manter um contador local, incrementando esse valor ao enviar mensagens e atualizando o relógio ao receber mensagens de outros nós.
4. Depois disso, foi implementada a exclusão mútua distribuída com o algoritmo de Ricart-Agrawala. Foram adicionadas mensagens do tipo REQUEST e REPLY, além de variáveis para controlar se o nó estava tentando entrar na região crítica, se já estava dentro dela, quais respostas já tinha recebido e quais respostas precisavam ser adiadas.
5. Também foi implementada a regra de prioridade da exclusão mútua usando o par (timestamp, node_id). Assim, quando dois nós pedem a região crítica, o menor timestamp tem prioridade e, em caso de empate, vence o menor ID.
6. Por fim, foi implementada a eleição de líder com o algoritmo Bully. Para isso, foram adicionadas mensagens ELECTION, OK e COORDINATOR, além dos comandos para consultar o líder atual e iniciar uma eleição manualmente.
7. A falha do líder foi simulada encerrando o processo do nó líder com CTRL + C. Após isso, outro nó pôde iniciar uma eleição e assumir como novo líder caso não encontrasse nenhum nó de ID maior ativo.
8. Ao longo dos testes, também foram feitas correções de implementação, principalmente em problemas de indentação, reinicialização dos processos após mudanças no código e ajustes na comunicação entre os nós.

Portanto, o código final foi escrito, testado e ajustado durante o desenvolvimento do trabalho. O ChatGPT foi usado como apoio para explicação, organização e depuração, mas a implementação final foi adaptada, executada e validada manualmente para este projeto.

Conclusão

Este trabalho ajudou a colocar em prática alguns conceitos importantes de sistemas distribuídos.

O relógio lógico de Lamport foi usado para ordenar eventos entre processos independentes. O algoritmo de Ricart-Agrawala foi usado para controlar o acesso à região crítica. O algoritmo Bully foi usado para eleger um novo líder após a falha simulada do líder anterior.

A comunicação por sockets TCP permitiu que os processos trocassem mensagens de verdade, mesmo rodando localmente na mesma máquina.

Com os testes realizados, foi possível observar o funcionamento dos algoritmos e validar a implementação proposta.